

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

**BHUMIKA U
(1BM24CS073)**

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by **BHUMIKA U (1BM24CS073)** who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Ms. Pallavi C V
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	1
2	Implement Johnson Trotter algorithm to generate permutations.	3
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	6
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	9
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	12
6	Implement 0/1 Knapsack problem using dynamic programming.	15
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	18
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	21
9	Implement Fractional Knapsack using Greedy technique.	26
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	29
11	Implement "N-Queens Problem" using Backtracking.	32
12	LeetCode Problems	36

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>

#define MAX 10

int graph[MAX][MAX], visited[MAX], stack[MAX];

int n, top = -1;

void dfs(int v) {
    visited[v] = 1;
    for (int i = 0; i < n; i++) {
        if (graph[v][i] && !visited[i]) {
            dfs(i);
        }
    }
    stack[++top] = v;
}

void topologicalSort() {
    for (int i = 0; i < n; i++)
        if (!visited[i])
            dfs(i);

    printf("Topological Ordering: ");
    while (top != -1)
        printf("%d ", stack[top--]);
}

int main() {
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix row by row:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
```

```

        printf("graph[%d][%d]: ", i, j);

        scanf("%d", &graph[i][j]);

    }

    visited[i] = 0;

}

topologicalSort();

}

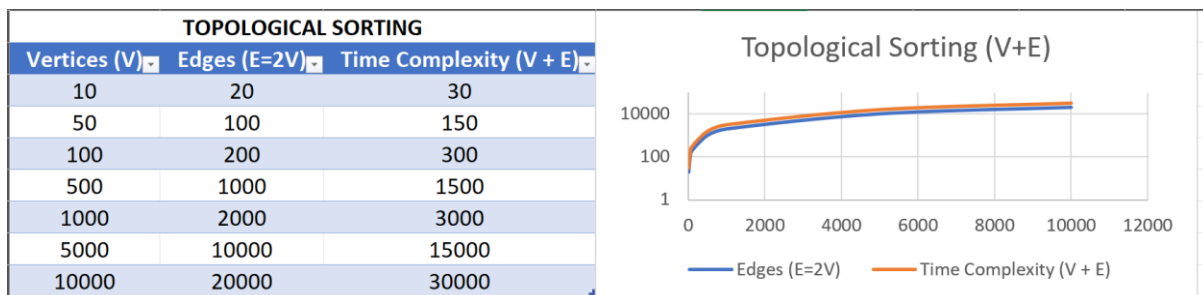
```

Output:

```

Enter number of vertices: 3
Enter adjacency matrix row by row:
graph[0][0]: 1
graph[0][1]: 1
graph[0][2]: 0
graph[1][0]: 0
graph[1][1]: 0
graph[1][2]: 0
graph[2][0]: 0
graph[2][1]: 1
graph[2][2]: 0
Topological Ordering: 2 0 1

```



Lab Program 2:

Implement Johnson Trotter algorithm to generate permutations.

```
#include <stdio.h>

int main() {

    int n;

    printf("Enter number of elements: ");

    scanf("%d", &n);

    int p[n], dir[n];

    for(int i = 0; i < n; i++) {

        p[i] = i + 1;

        dir[i] = -1; // -1 = left, 1 = right

    }

    for(int i = 0; i < n; i++)

        printf("%d ", p[i]);

    printf("\n");

    while(1) {

        int mobile = 0, pos;

        for(int i = 0; i < n; i++) {

            if(dir[p[i]-1] == -1 && i > 0 && p[i] > p[i-1] && p[i] > mobile) {

                mobile = p[i];

                pos = i;

            }

            if(dir[p[i]-1] == 1 && i < n-1 && p[i] > p[i+1] && p[i] > mobile) {

                mobile = p[i];

                pos = i;

            }

        }

        if(mobile == 0)

            break;

    }

}
```

```
if(dir[mobile-1] == -1) {  
    int temp = p[pos];  
    p[pos] = p[pos-1];  
    p[pos-1] = temp;  
    pos--;  
} else {  
    int temp = p[pos];  
    p[pos] = p[pos+1];  
    p[pos+1] = temp;  
    pos++;  
}  
for(int i = 0; i < n; i++)  
    if(p[i] > mobile)  
        dir[p[i]-1] *= -1;  
for(int i = 0; i < n; i++)  
    printf("%d ", p[i]);  
printf("\n");  
}  
return 0;  
}
```

Output:

```
Enter number of elements: 3
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3
```


Lab Program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

```
#include <stdio.h>

#include <time.h>

int main() {

    int low, mid, high;

    printf("Enter low: ");

    scanf("%d",&low);

    printf("Enter mid: ");

    scanf("%d",&mid);

    printf("Enter high: ");

    scanf("%d",&high);

    int a[100], b[100];

    printf("Enter %d sorted elements:\n", (high-low)+1);

    for(int i=low;i<=high;i++){

        scanf("%d",&a[i]);

    }

    clock_t start, end;

    double cpu_time_used;

    start = clock();

    int i = low;

    int j = mid + 1;

    int k = low;

    while(i<=mid && j<=high){

        if(a[i] <= a[j]){

            b[k] = a[i];

            i++;

        }

        else{
```

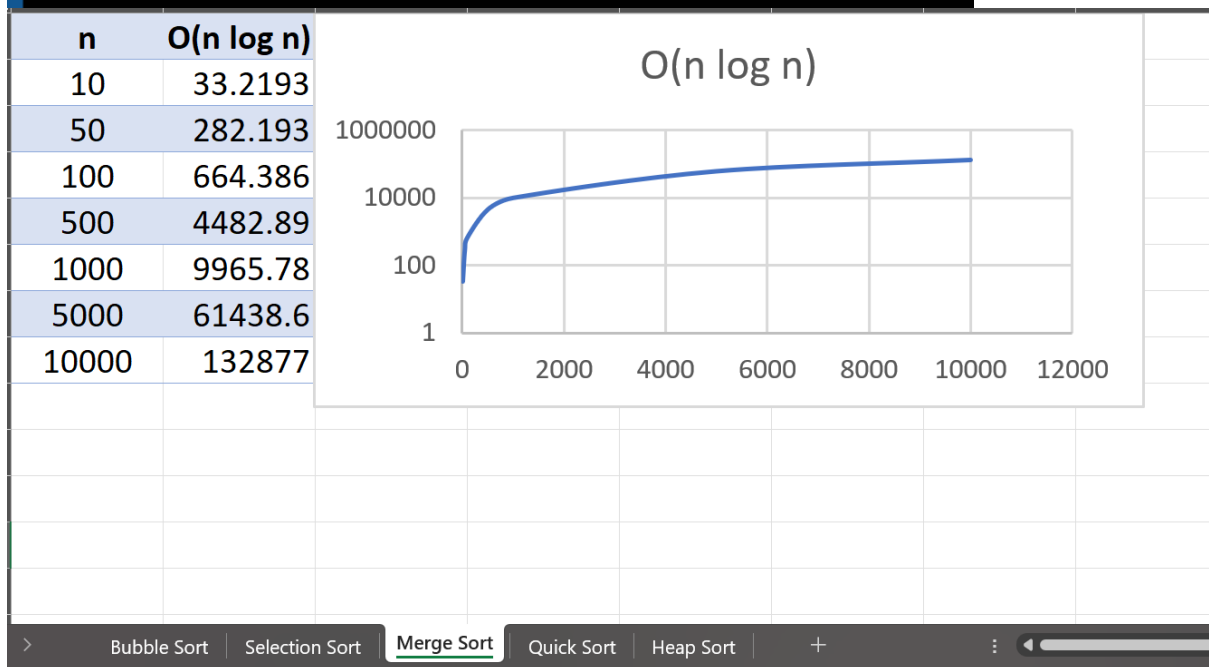
```

        b[k] = a[j];
        j++;
    }
    k++;
}
while(i<=mid){
    b[k] = a[i];
    i++;
    k++;
}
while(j<=high){
    b[k] = a[j];
    j++;
    k++;
}
end = clock();
cpu_time_used = ((double) (end - start)) / CLOCKS_PER_SEC;
printf("Merged array:\n");
for(k=low;k<=high;k++){
    printf("%d ", b[k]);
}
printf("\nExecution time: %f seconds\n", cpu_time_used);
return 0;
}

```

Output:

```
Enter low: 2
Enter mid: 5
Enter high: 7
Enter 6 sorted elements:
2 5 8 1 6 9
Merged array:
2 5 6 8 1 9
Execution time: 0.000001 seconds
```



Lab Program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

```
#include <stdio.h>

#include <time.h>

int partition(int a[100], int low, int high){
    int i, j, key, temp;

    i = low + 1;
    j = high;
    key = a[low];
    while(1){
        while(a[i] <= key && i < high) i++;
        while(a[j] > key) j--;
        if(i < j){
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
        else{
            temp = a[low];
            a[low] = a[j];
            a[j] = temp;
            return j;
        }
    }
}

void quicksort(int a[100], int low, int high){
    int j;
    if(low < high){
        j = partition(a, low, high);
```

```

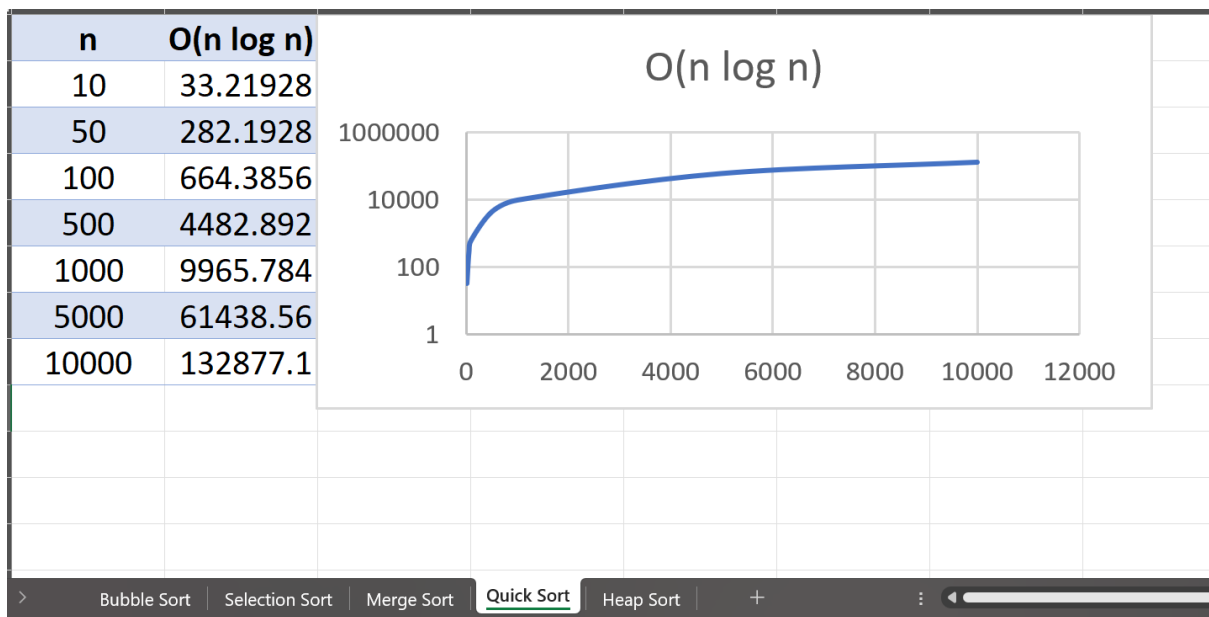
        quicksort(a, low, j-1);
        quicksort(a, j+1, high);
    }
}

int main(){
    int low, high, i;
    clock_t start, end;
    double time_taken;
    printf("Enter low: ");
    scanf("%d",&low);
    printf("Enter high: ");
    scanf("%d",&high);
    int a[100];
    printf("Enter %d elements:\n",(high - low) + 1);
    for(i = low; i <= high; i++){
        scanf("%d",&a[i]);
    }
    start = clock();
    quicksort(a, low, high);
    end = clock();
    printf("Sorted array:\n");
    for(i = low; i <= high; i++){
        printf("%d ", a[i]);
    }
    time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("\nTime taken = %f seconds", time_taken);
    return 0;
}

```

Output:

```
Enter low: 2 5
Enter high: Enter 4 elements:
1 9 8 7
Sorted array:
1 7 8 9
Time taken = 0.000001 seconds
```



Lab Program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

void heapify(int a[], int n, int i)
{
    int c, item;
    int p=i;
    item=a[p];
    c=2*p+1;
    while(c<=n-1){
        if(c+1<=n-1){
            if(a[c]<a[c+1]) c++;
        }
        if(item<a[c]){
            a[p]=a[c];
            p=c;
            c=2*p+1;
        }
        else break;
    }
    a[p]=item;
}

void buildheap(int a[],int n){
    int i;
    for(i=(n-1)/2;i>=0;i--){
        heapify(a,n,i);
    }
}
```

```

}

void heapSort(int a[], int n)
{
    int i, temp;
    buildheap(a,n);
    for(i=n-1;i>0;i--){
        temp=a[0];
        a[0]=a[i];
        a[i]=temp;
        heapify(a,i,0);
    }
}

int main()
{
    int n;
    clock_t start, end;
    double cpu_time;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    start = clock();
    heapSort(a, n);
    end = clock();
    cpu_time = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nSorted elements are:\n");
    for (int i = 0; i < n; i++)

```



```

    printf("%d ", a[i]);

    printf("\n\nTime taken = %f seconds\n", cpu_time);

    return 0;
}

```

Output:

```

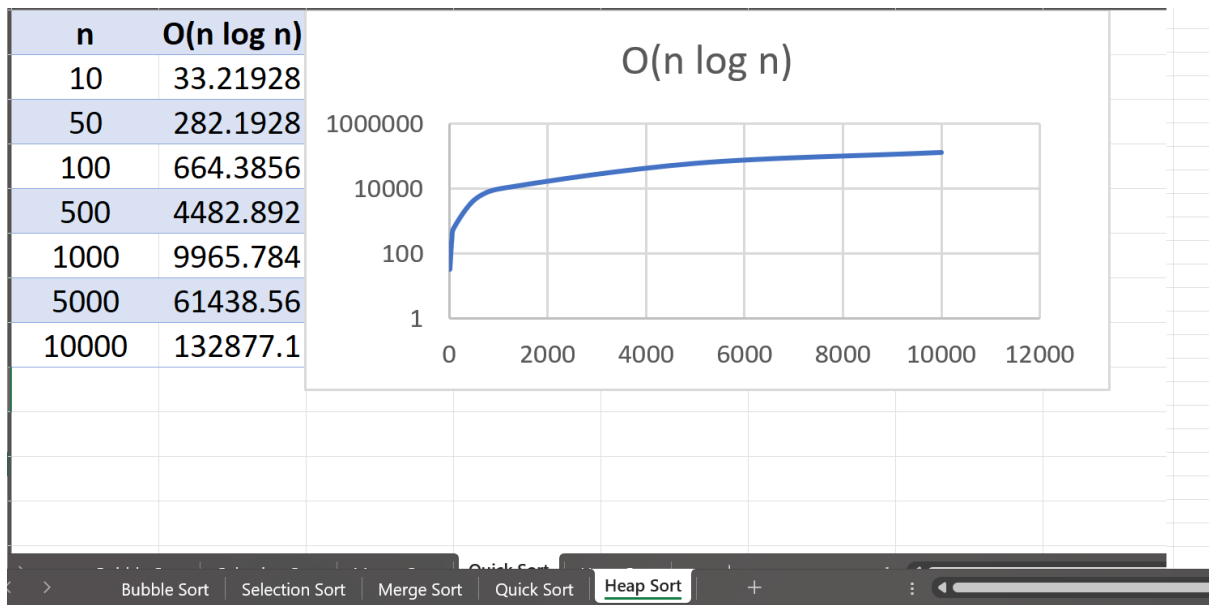
C:\Users\BMSCECSE-L3-17\De X + v
Enter number of elements: 8
Enter elements:
15
11
8
30
70
20
2
12

Sorted elements are:
2 8 11 12 15 20 30 70

Time taken = 0.000000 seconds

Process returned 0 (0x0)   execution time : 8.491 s
Press any key to continue.

```



Lab Program 6:

Implement 0/1 Knapsack problem using dynamic programming.

```
#include<stdio.h>

#include<time.h>

int maximum(int x, int y){
    if(x>y){
        return x;
    }
    else{
        return y;
    }
}

int KnapsackDP(int M,int n,int W[],int P[]){
    int Table[n+1][M+1];
    for(int i=0;i<n;i++){
        Table[i][0]=0;
    }
    for(int j=0;j<=M;j++){
        Table[0][j]=0;
    }
    for(int i=1;i<=n;i++){
        for(int j=1;j<=M;j++){
            if(j<W[i-1]){
                Table[i][j]=Table[i-1][j];
            }
            else{
                Table[i][j] = maximum( Table[i-1][j], P[i-1] + Table[i-1][j - W[i-1]]);
            }
        }
    }
}
```

```

    }

    return Table[n][M];
}

int main() {
    int n;

    printf("Enter the number of objects: ");
    scanf("%d",&n);

    int M;

    printf("Enter the size of the Knapsack: ");
    scanf("%d",&M);

    int W[n], P[n];

    printf("Enter the Weights and their respective Profits: \n");
    for(int i=0;i<n;i++){
        scanf("%d %d",&W[i], &P[i]);
    }

    clock_t start,end;

    start = clock();

    int res=KnapsackDP(M,n,W,P);

    end=clock();

    float time=(double)(end-start)*1000/CLOCKS_PER_SEC;

    printf("The maximum profit obtained is: %d",res);

    printf("\nTime: %.2f", time);

    return 0;
}

```

Output:

```
Enter the number of objects: 5
Enter the size of the Knapsack: 10
Enter the Weights and their respective Profits:
2 20
3 30
4 40
5 50
9 100
The maximum profit obtained is: 100
Time: 0.00
```

Lab Program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include <stdio.h>

#include <time.h>

#define MAX 20

#define INF 9999

void floyds(int n, int cost[MAX][MAX], int A[MAX][MAX]) {
    int i, j, k;

    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            A[i][j] = cost[i][j];
        }
    }

    for(k = 0; k < n; k++) {
        for(i = 0; i < n; i++) {
            for(j = 0; j < n; j++) {
                if(A[i][k] + A[k][j] < A[i][j]) {
                    A[i][j] = A[i][k] + A[k][j];
                }
            }
        }
    }
}

int main() {
    int n;

    int cost[MAX][MAX], A[MAX][MAX];

    int i, j;

    clock_t start, end;

    double cpu_time_used;
```

```

printf("Enter number of vertices: ");
scanf("%d", &n);
printf("Enter the cost matrix:\n");
printf("(Enter %d for INF / no direct edge)\n", INF);
for(i = 0; i < n; i++) {
    for(j = 0; j < n; j++) {
        scanf("%d", &cost[i][j]);
    }
}
start = clock();
floyds(n, cost, A);
end = clock();
cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("\nShortest Distance Matrix:\n");
for(i = 0; i < n; i++) {
    for(j = 0; j < n; j++) {
        if(A[i][j] == INF)
            printf("INF\t");
        else
            printf("%d\t", A[i][j]);
    }
    printf("\n");
}
printf("\nExecution Time: %f seconds\n", cpu_time_used);
return 0;
}

```

Output:

```
Enter number of vertices: 4
Enter the cost matrix:
(Enter 9999 for INF / no direct edge)
0 3 9999 7
8 0 2 9999
5 9999 0 1
2 9999 9999 0

Shortest Distance Matrix:
0      3      5      6
5      0      2      3
3      6      0      1
2      5      7      0

Execution Time: 0.000000 seconds

Process returned 0 (0x0)   execution time : 39.910 s
Press any key to continue.
```

Lab Program 8:

- a. Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
#include <stdio.h>
#define INF 9999
#define MAX 10
void prims(int n, int cost[MAX][MAX]) {
    int near[MAX], t[MAX][2];
    int i, j, k, l, min, mincost = 0;
    min = INF;
    for (i = 1; i <= n; i++) {
        for (j = i + 1; j <= n; j++) {
            if (cost[i][j] < min) {
                min = cost[i][j];
                k = i;
                l = j;
            }
        }
    }
    t[1][1] = k;
    t[1][2] = l;
    mincost = cost[k][l];
    for (i = 1; i <= n; i++) {
        if (cost[i][k] < cost[i][l])
            near[i] = k;
        else
            near[i] = l;
    }
    near[k] = 0;
    near[l] = 0;
    for (i = 2; i <= n - 1; i++) {
        min = INF;
        for (j = 1; j <= n; j++) {
            if (near[j] != 0 && cost[j][near[j]] < min) {
                min = cost[j][near[j]];
                k = j;
            }
        }
        t[i][1] = k;
        t[i][2] = near[k];
        mincost += cost[k][near[k]];
        near[k] = 0;
        for (j = 1; j <= n; j++) {
            if (near[j] != 0 && cost[j][near[j]] > cost[j][k]) {
                near[j] = k;
            }
        }
    }
    printf("\nEdges in MST:\n");
    for (i = 1; i <= n - 1; i++) {
        printf("%d - %d\n", t[i][1], t[i][2]);
    }
    printf("Minimum cost = %d\n", mincost);
}
int main() {
```



```

int n, cost[MAX][MAX];
int i, j;
printf("Enter number of vertices: ");
scanf("%d", &n);
printf("Enter cost matrix:\n");
for (i = 1; i <= n; i++) {
    for (j = 1; j <= n; j++) {
        scanf("%d", &cost[i][j]);
        if (cost[i][j] == 0)
            cost[i][j] = INF;
    }
}
prims(n, cost);
return 0;
}

```

Output:

```

Enter number of vertices: 4
Enter cost matrix:
0 1 3 2
1 0 4 0
3 4 0 5
2 0 5 0

Edges in MST:
1 - 2
4 - 1
3 - 1
Minimum cost = 6

```

b. Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
#include <stdio.h>

#define MAX 10
#define INF 9999

int parent[MAX];

int find(int i) {
    while (parent[i])
        i = parent[i];
    return i;
}

void union_set(int i, int j) {
    parent[j] = i;
}

int main() {
    int n, cost[MAX][MAX];
    int t[MAX][2];
    int i, j, u, v, k;
    int min, mincost = 0;
    int edge_count = 0;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter cost matrix:\n");
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0)
                cost[i][j] = INF;
        }
    }
    for (i = 1; i <= n; i++)
        parent[i] = 0;
    while (edge_count < n - 1) {
        min = INF;
```

```

    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            if (cost[i][j] < min) {
                min = cost[i][j];
                u = i;
                v = j;
            }
        }
    }
    i = find(u);
    j = find(v);
    if (i != j) {
        edge_count++;
        t[edge_count][1] = u;
        t[edge_count][2] = v;
        mincost += min;
        union_set(i, j);
    }
    cost[u][v] = cost[v][u] = INF;
}
printf("\nEdges in MST:\n");
for (i = 1; i <= edge_count; i++) {
    printf("%d - %d\n", t[i][1], t[i][2]);
}
printf("Minimum cost = %d\n", mincost);
return 0;
}

```

Output:

```
Enter number of vertices: 4
```

```
Enter cost matrix:
```

```
1 0 3 4
```

```
1 0 4 0
```

```
3 4 0 5
```

```
2 0 5 0
```

```
Edges in MST:
```

```
2 - 1
```

```
4 - 1
```

```
1 - 3
```

```
Minimum cost = 6
```

Lab Program 9:

Implement Fractional Knapsack using Greedy technique.

```
#include<stdio.h>

#include<time.h>

struct Item{
    int weight;
    int profit;
    float ratio;
};

void sort(struct Item items[], int n){
    for(int i=0;i<n-1;i++){
        for(int j=i+1;j<n;j++){
            if(items[i].ratio < items[j].ratio){
                struct Item temp=items[i];
                items[i]=items[j];
                items[j]=temp;
            }
        }
    }
}

float fractionalKnapsack(int cap,struct Item items[], int n){
    int remcap=cap;
    float mincost=0.0;
    for(int i=0;i<n;i++){
        if(remcap >= items[i].weight){
            remcap-=items[i].weight;
            mincost+=items[i].profit;
        }
        else{
            mincost=mincost + (items[i].ratio * remcap);
            break;
        }
    }
    return mincost;
}
```

```

}
int main(){
    int n, cap;
    clock_t start,end;
    printf("Enter the number of items: ");
    scanf("%d",&n);
    struct Item items[n];
    printf("Enter the weight and profit of all the items: ");
    for(int i=0;i<n;i++){
        scanf("%d %d",&items[i].weight,&items[i].profit);
        items[i].ratio=(float)items[i].profit/items[i].weight;
    }
    printf("Enter the capacity of the Knapsack: ");
    scanf("%d",&cap);
    start=clock();
    sort(items,n);
    float cost=fractionalKnapsack(cap,items,n);
    end=clock();
    printf("The Optimal Solution is: %.2f",cost);
    double time_taken=(double)(end-start)*10000/CLOCKS_PER_SEC;
    printf("\nExecution time: %f seconds",time_taken);
    return 0;
}

```

Output:

```
Enter the number of items: 3
Enter the weight and profit of all the items: 10 60
20 100
30 120
Enter the capacity of the Knapsack: 0
The Optimal Solution is: 0.00
Execution time: 0.010000 seconds
```

Lab Program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include <stdio.h>

#include <limits.h>

#include <time.h>

#define MAX 100

int minDistance(int dist[], int visited[], int n) {

    int min = INT_MAX, min_index = -1;

    for (int i = 0; i < n; i++) {

        if (visited[i] == 0 && dist[i] < min) {

            min = dist[i];

            min_index = i;

        }

    }

    return min_index;

}

void dijkstra(int graph[MAX][MAX], int n, int src) {

    int dist[MAX], visited[MAX];

    for (int i = 0; i < n; i++) {

        dist[i] = INT_MAX;

        visited[i] = 0;

    }

    dist[src] = 0;

    for (int count = 0; count < n - 1; count++) {

        int u = minDistance(dist, visited, n);

        visited[u] = 1;

        for (int v = 0; v < n; v++) {

            if (visited[v] == 0 &&

                graph[u][v] > 0 &&
```



```

        dist[u] != INT_MAX &&
        dist[u] + graph[u][v] < dist[v]) {
            dist[v] = dist[u] + graph[u][v];
        }
    }
}

printf("\nMinimum distances from source %d:\n", src);
for (int i = 0; i < n; i++) {
    printf("To %d = %d\n", i, dist[i]);
}
}

int main() {
    int n, graph[MAX][MAX], src;
    clock_t start, end;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter weighted adjacency matrix (0 = no edge):\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }
    printf("Enter source vertex: ");
    scanf("%d", &src);
    start=clock();
    dijkstra(graph, n, src);
    end=clock();
    float time_taken=(double)(end-start)*1000/CLOCKS_PER_SEC;
    printf("Execution time: %f seconds",time_taken);
}

```

```
    return 0;  
}
```

Output:

```
Enter number of vertices: 4  
Enter weighted adjacency matrix (0 = no edge):  
0 1 3 2  
1 0 4 0  
3 4 0 4  
2 0 5 0  
Enter source vertex: 0  
  
Minimum distances from source 0:  
To 0 = 0  
To 1 = 1  
To 2 = 3  
To 3 = 2  
Execution time: 0.023000 seconds
```

Lab Program 11:

Implement “N-Queens Problem” using Backtracking.

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

#define MAX 20

int board[MAX][MAX];

int N;

void printBoard() {
    int i, j;

    printf("\nSolution:\n\n");

    for(i = 0; i < N; i++) {
        for(j = 0; j < N; j++) {
            if(board[i][j] == 1)
                printf(" Q ");
            else
                printf(" . ");
        }
        printf("\n");
    }
}

int isSafe(int row, int col) {
    int i, j;

    for(i = 0; i < row; i++) {
        if(board[i][col] == 1)
            return 0;
    }

    for(i = row - 1, j = col - 1;
        i >= 0 && j >= 0;
```

```

        i--, j--) {
            if(board[i][j] == 1)
                return 0;
        }
        for(i = row - 1, j = col + 1;
            i >= 0 && j < N;
            i--, j++) {
                if(board[i][j] == 1)
                    return 0;
            }
        return 1;
    }
}

int solveNQueens(int row) {
    if(row == N)
        return 1;

    int col;
    for(col = 0; col < N; col++) {
        if(isSafe(row, col)) {
            board[row][col] = 1;
            if(solveNQueens(row + 1))
                return 1;
            board[row][col] = 0;
        }
    }
    return 0;
}

int main() {
    int i, j;

    printf("Enter value of N: ");

```

```

scanf("%d", &N);
for(i = 0; i < N; i++) {
    for(j = 0; j < N; j++) {
        board[i][j] = 0;
    }
}
clock_t start, end;
start = clock();
if(solveNQueens(0)) {
    printBoard();
}
else {
    printf("No solution exists.\n");
}
end = clock();
double time_taken =
    ((double)(end - start)) / CLOCKS_PER_SEC;
printf("\nExecution Time: %f seconds\n", time_taken);
return 0;
}

```

Output:

```
Enter value of N: 4
```

```
Solution:
```

```
.  Q  .  .  
.  .  .  Q  
Q  .  .  .  
.  .  Q  .
```

```
Execution Time: 0.001000 seconds
```

```
Process returned 0 (0x0)   execution time : 13.368 s  
Press any key to continue.  
|
```

LEETCODE PROGRAMS:

11. Container with the most water

The screenshot displays the LeetCode interface for problem 11. The left pane shows the submission history with the following details:

Status	Language	Runtime	Memory	Notes
Accepted Feb 23, 2026	C	0 ms	14.1 MB	
Time Limit Exceeded Feb 23, 2026	C	N/A	N/A	
Time Limit Exceeded Feb 23, 2026	C	N/A	N/A	

The right pane shows the C code for the solution:

```
1 int maxArea(int* height, int heightSize) {
2     int left = 0;
3     int right = heightSize - 1;
4     int max = 0;
5
6     while (left < right) {
7         int width = right - left;
8         int h;
9
10        if (height[left] < height[right]) {
11            h = height[left];
12            left++;
13        } else {
14            h = height[right];
15            right--;
16        }
17
18        int area = h * width;
19
20        if (area > max) {
21            max = area;
22        }
23    }
24 }
```

34. Find first and last position of an element in a sorted array

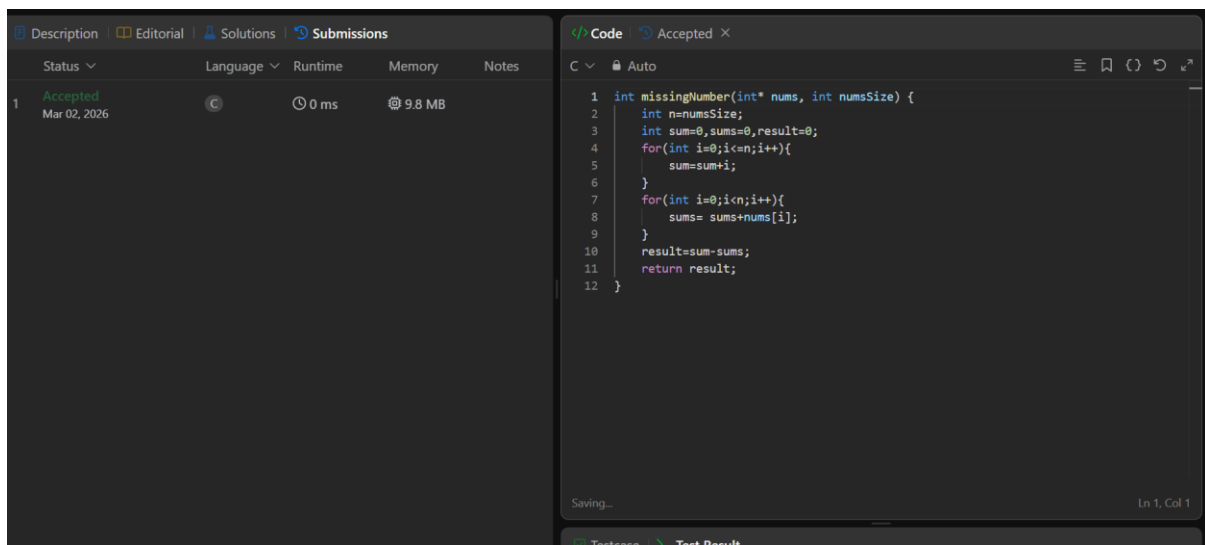
The screenshot displays the LeetCode interface for problem 34. The left pane shows the submission history with the following details:

Status	Language	Runtime	Memory	Notes
Accepted Feb 27, 2026	C	0 ms	10.1 MB	

The right pane shows the C code for the solution:

```
1 /**
2  * Note: The returned array must be malloced, assume caller calls free().
3  */
4 int* searchRange(int* nums, int numsSize, int target, int* returnSize) {
5     int* result = (int*)malloc(2 * sizeof(int)); (start and end)
6     *returnSize = 2;
7
8     int start = -1, end = -1;
9
10    for (int i = 0; i < numsSize; i++) {
11        if (nums[i] == target) {
12            if (start == -1) start = i;
13            end = i;
14        }
15    }
16
17    result[0] = start;
18    result[1] = end;
19
20    return result;
21 }
22 }
```

268. Missing Number

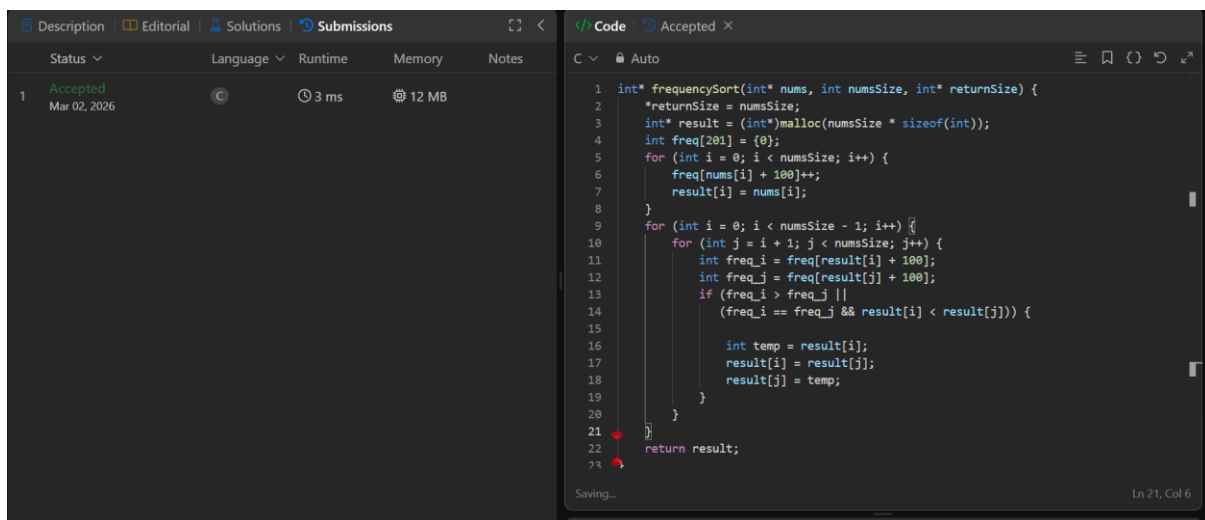


The screenshot shows a code editor with the following C code:

```
1 int missingNumber(int* nums, int numsSize) {
2     int n=numsSize;
3     int sum=0,sums=0,result=0;
4     for(int i=0;i<n;i++){
5         sum=sum+i;
6     }
7     for(int i=0;i<n;i++){
8         sums= sums+nums[i];
9     }
10    result=sum-sums;
11    return result;
12 }
```

The submission status is "Accepted" on Mar 02, 2026, with a runtime of 0 ms and memory usage of 9.8 MB.

1636. Sort an array by Increasing frequency



The screenshot shows a code editor with the following C code:

```
1 int* frequencySort(int* nums, int numsSize, int* returnSize) {
2     *returnSize = numsSize;
3     int* result = (int*)malloc(numsSize * sizeof(int));
4     int freq[201] = {0};
5     for (int i = 0; i < numsSize; i++) {
6         freq[nums[i] + 100]++;
7         result[i] = nums[i];
8     }
9     for (int i = 0; i < numsSize - 1; i++) {
10        for (int j = i + 1; j < numsSize; j++) {
11            int freq_i = freq[result[i] + 100];
12            int freq_j = freq[result[j] + 100];
13            if (freq_i > freq_j ||
14                (freq_i == freq_j && result[i] < result[j])) {
15                int temp = result[i];
16                result[i] = result[j];
17                result[j] = temp;
18            }
19        }
20    }
21    return result;
22 }
```

The submission status is "Accepted" on Mar 02, 2026, with a runtime of 3 ms and memory usage of 12 MB.

207. Course Schedule

DescriptionEditorialSolutionsSubmissions

Status	Language	Runtime	Memory	Notes
2 Accepted Mar 02, 2026	C	41 ms	125.8 MB	
1 Accepted Mar 02, 2026	C	35 ms	125.7 MB	

CodeAccepted

```
1 bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int* prerequisitesColSize) {
2     int** graph = (int**)malloc(numCourses * sizeof(int*));
3     int* indegree = (int*)calloc(numCourses, sizeof(int));
4     int* graphColSize = (int*)calloc(numCourses, sizeof(int));
5     for (int i = 0; i < numCourses; i++) {
6         graph[i] = (int*)malloc(prerequisitesSize * sizeof(int));
7     }
8     for (int i = 0; i < prerequisitesSize; i++) {
9         int a = prerequisites[i][0];
10        int b = prerequisites[i][1];
11        graph[b][graphColSize[b]++] = a;
12        indegree[a]++;
13    }
14    int* queue = (int*)malloc(numCourses * sizeof(int));
15    int front = 0, rear = 0;
16    for (int i = 0; i < numCourses; i++) {
17        if (indegree[i] == 0) {
18            queue[rear++] = i;
19        }
20    }
21    int count = 0;
22    while (front < rear) {
```

801. Minimum Swaps to make sequence Increasing

801. Minimum Swaps To Make Sequences Increasing Solved

HardTopicsCompanies

You are given two integer arrays of the same length `nums1` and `nums2`. In one operation, you are allowed to swap `nums1[i]` with `nums2[i]`.

- For example, if `nums1 = [1,2,3,8]`, and `nums2 = [5,6,7,4]`, you can swap the element at `i = 3` to obtain `nums1 = [1,2,3,4]` and `nums2 = [5,6,7,8]`.

Return the **minimum number of needed operations to make `nums1` and `nums2` strictly increasing**. The test cases are generated so that the given input always makes it possible.

An array `arr` is **strictly increasing** if and only if `arr[0] < arr[1] < arr[2] < ... < arr[arr.length - 1]`.

Example 1:

Input: `nums1 = [1,3,5,4], nums2 = [1,2,3,7]`

CodeAuto

```
1 #include <limits.h>
2 int min(int a, int b) {
3     return (a < b) ? a : b;
4 }
5 int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size) {
6     int keep = 0;
7     int swap = 1;
8     for (int i = 1; i < nums1Size; i++) {
9         int newKeep = INT_MAX;
10        int newSwap = INT_MAX;
11        if (nums1[i] > nums1[i - 1] && nums2[i] > nums2[i - 1]) {
12            newKeep = min(newKeep, keep);
13            newSwap = min(newSwap, swap);
14        }
15        if (nums1[i] > nums2[i - 1] && nums2[i] > nums1[i - 1]) {
16            newKeep = min(newKeep, swap);
17            newSwap = min(newSwap, keep + 1);
18        }
19        keep = newKeep;
20        swap = newSwap;
21    }
22    return min(keep, swap);
23 }
```

287. Find the duplicate Number

DescriptionEditorialSolutionsSubmissions

Status	Language	Runtime	Memory	Notes
1 Accepted Apr 16, 2026	C	4 ms	17.3 MB	

CodeAuto

```
1 #include <stdlib.h>
2
3 int findDuplicate(int* nums, int n) {
4     int* seen = calloc(n, sizeof(int));
5
6     for (int i = 0; i < n; i++) {
7         if (seen[nums[i]]) {
8             free(seen);
9             return nums[i];
10        }
11        seen[nums[i]] = 1;
12    }
13
14    free(seen);
15    return -1;
16 }
```